

# 오류 검출과 정정

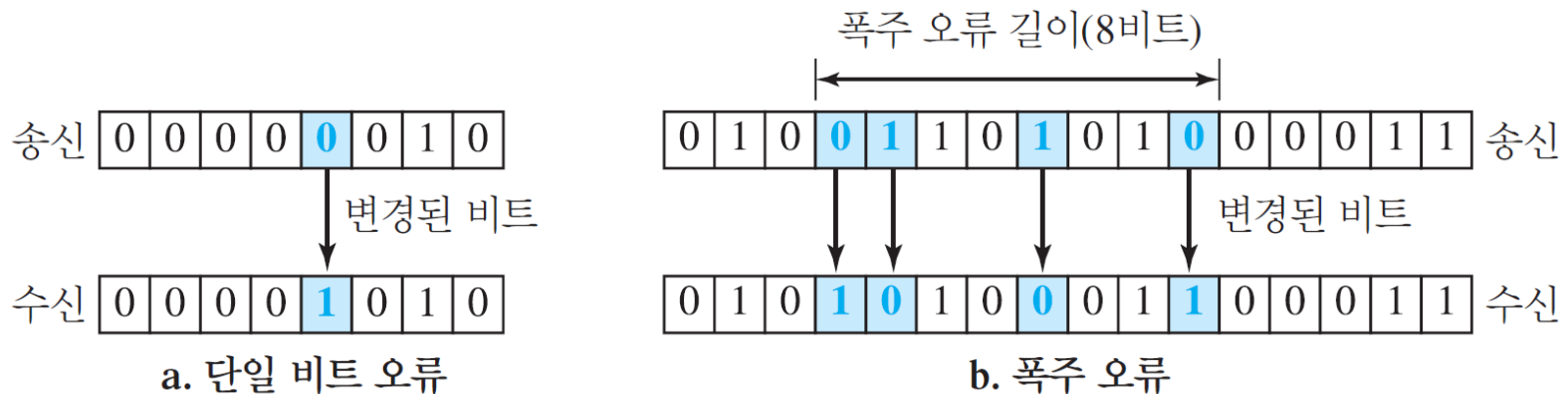
# 목 차

- 개요
- 블록 부호화
- 순환 코드
- 검사합

# 개요

## ■ 오류의 유형

- ▶ 단일-비트 오류(Single-Bit Error)
  - 데이터 단위 중 오직 하나의 비트만이  $1 \rightarrow 0$ ,  $0 \rightarrow 1$ 로 변경되는 오류를 의미
- ▶ 폭주 오류(Burst Error)
  - 데이터 단위에서 2개 이상의 연속적인 비트들이  $1 \rightarrow 0$ ,  $0 \rightarrow 1$ 로 바뀌는 오류를 의미



# 개요

## ■ 중복

- ▶ redundancy, 잉여 또는 덧붙임
- ▶ 오류를 검출하거나 정정하기 위해 데이터 이외에 추가하는 비트
- ▶ 중복 비트들은 송신자에 의해 더해지며 수신자가 제거
- ▶ 중복 비트들로 인해 수신자는 오류를 검출하거나 정정할 수 있음

# 개요

## ■ 검출 대 정정

- ▶ 오류 검출(error detection)
  - 오류가 있는지만 알아내는 것
- ▶ 오류 정정(error correction)
  - 정확하게 몇 비트가 잘못되었는지 알아야 하며, 더욱이 어디가 잘못되었는지를 알아야 내는 것
- ▶ 오류를 정정하는 것은 검출하는 것보다 어렵다.

# 개요

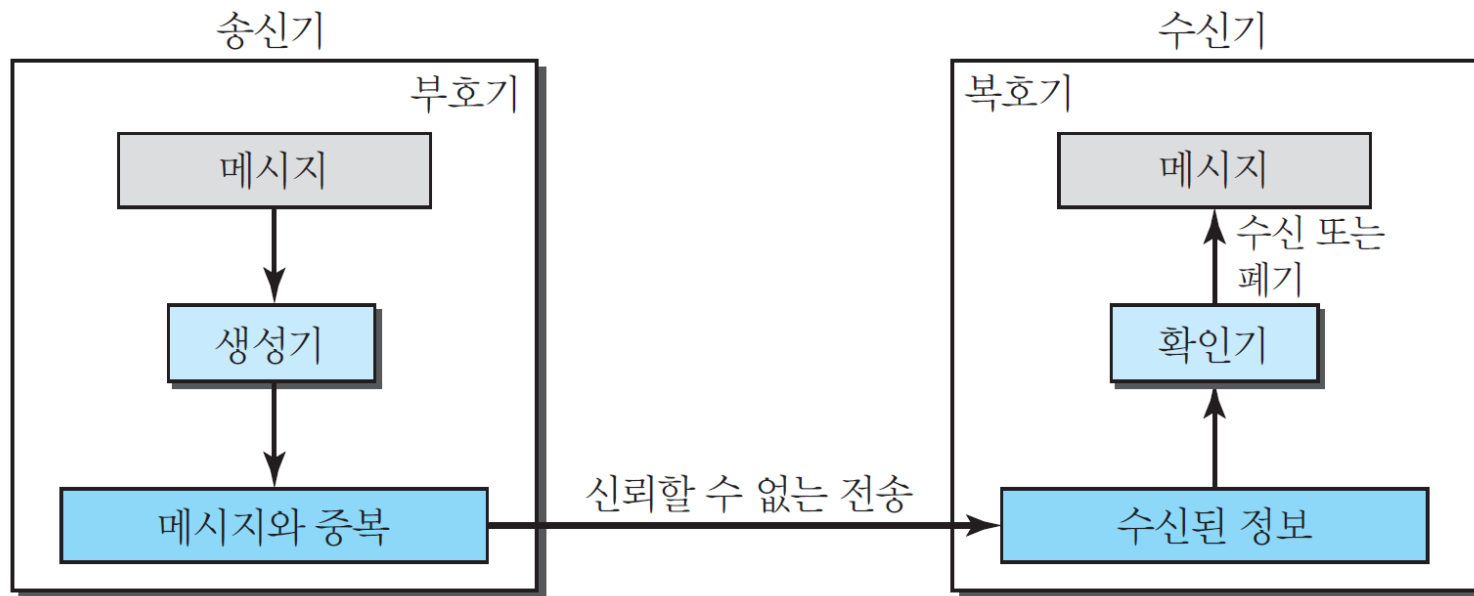
## ■ 전진 오류 정정 대 재전송

- ▶ 전진 오류 정정(forward error correction)
  - 수신자가 메시지의 중복 비트를 이용하여 메시지를 추측해가는 과정
- ▶ 재전송(retransmission)
  - 오류 발생을 검출하여 송신자에게 다시 보내줄 것을 요청하는 과정

# 개요

## ■ 부호화(coding)

- ▶ 블록 부호화(block coding)
- ▶ 컨볼루션 부호화 부호화(convolution coding)



# 개요

- ▶ 모듈러 연산(modular arithmetic)
  - 모듈러(modulus) N이라는 제한된 정수를 사용
  - 0부터 N-1까지 정수 사용
  - 모듈러-2 연산
    - ✓ 모듈러 N은 2, 0과 1만을 사용

adding     $0 + 0 = 0$     $0 + 1 = 1$     $1 + 0 = 1$     $1 + 1 = 0$   
subtracting    $0 - 0 = 0$     $0 - 1 = 1$     $1 - 0 = 1$     $1 - 1 = 0$

$$0 \oplus 0 = 0 \qquad 1 \oplus 1 = 0$$

a. 두 비트가 같으면 결과는 0

$$0 \oplus 1 = 1 \qquad 1 \oplus 0 = 1$$

b. 두 비트가 다르면 결과는 1

$$\begin{array}{r} \phantom{\oplus} 1 \phantom{0} 1 \phantom{1} 0 \\ \oplus 1 \phantom{0} 1 \phantom{0} 0 \\ \hline 0 \phantom{0} 1 \phantom{0} 1 \phantom{0} \end{array}$$

c. 두 비트 패턴의 XOR 결과



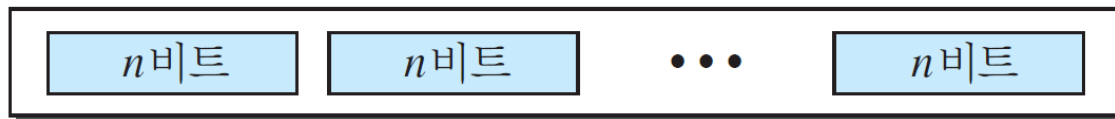
# 블록 부호화

## ■ 블록 부호화

- ▶ 메시지를 데이터워드(dataword)라는  $k$  비트의 블록으로 나눔
- ▶ 각 블록에  $r$ 개의 중복 비트들을 더하여 길이  $n = k + r$  로 만듦
- ▶  $n$  비트 블록을 코드워드(codeword)라고 부름



각  $k$ 비트로 된  $2^k$  데이터워드



각  $n$ 비트로 된  $2^n$  데이터워드

# 블록 부호화

## ▶ 예제 10.1

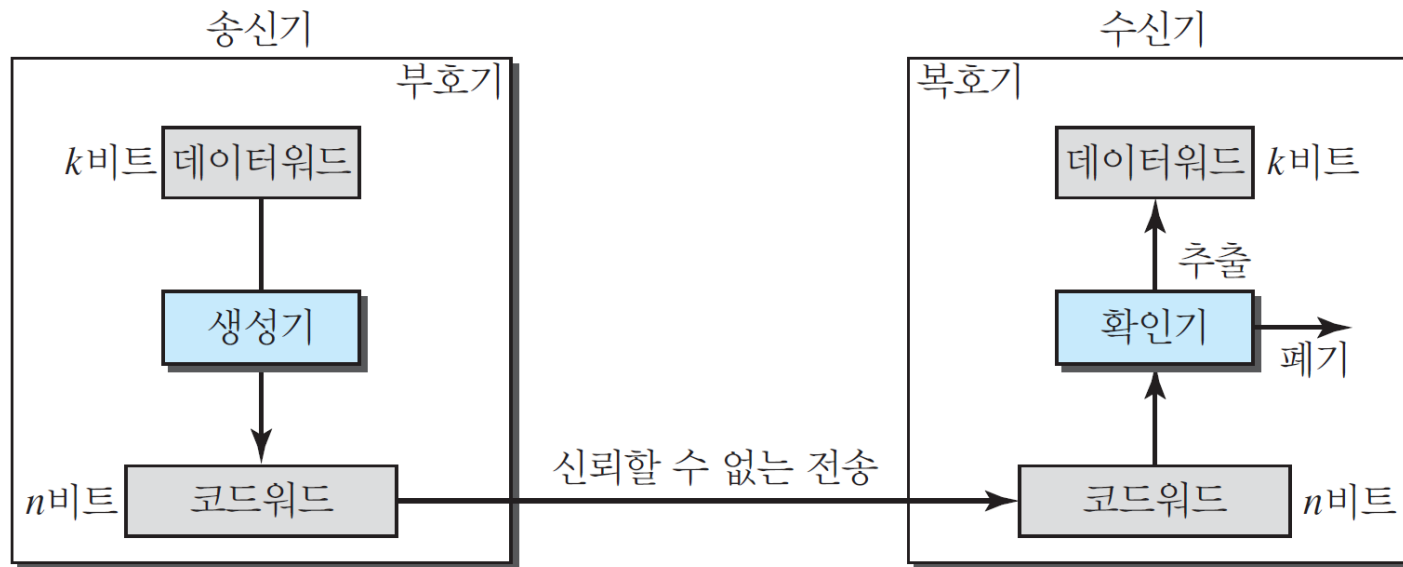
- 4장 디지털전송에서 4B/5B 부호화는 블록 부호화의 좋은 예
- 이 부호화 방법에서는  $k = 4$  이고  $n = 5$  이다.
- 모두  $2^k = 16$  개의 데이터워드와 있고  $2^n = 32$  개의 코드워드가 있다.
- 32개의 코드워드 중에서 16개가 메시지 전송에 사용되고 나머지는 다른 목적으로 이용되거나 사용되지 않는다.

# 블록 부호화

## ■ 오류 검출

### ▶ 오류 검출을 위한 조건

- 수신자는 유효 코드워드를 찾을 수 있거나 그 목록을 가지고 있다.
- 원래의 코드워드가 무효 코드워드로 바뀌었다.



# 블록 부호화

## ▶ 예제 10.2

- $k = 2$ 이고  $n = 3$ 이라고 하자.
- 송신자가 데이터워드 01을 011로 부호화하여 수신자에게 전송
- 수신자가 011을 수신, 이는 유효 코드, 데이터워드 01을 추출
- 코드워드가 전송 도중 손상되어 111이 수신된다면(맨 왼쪽 비트가 바뀌었다). 이는 유효 코드가 아니므로 버려진다.
- 코드워드가 000이 수신된다면(오른쪽 두 비트가 바뀌었다). 이는 유효코드, 이로부터 부정확한 데이터워드인 00을 추출. 2개 비트가 손상되어 오류를 찾지 못한다.

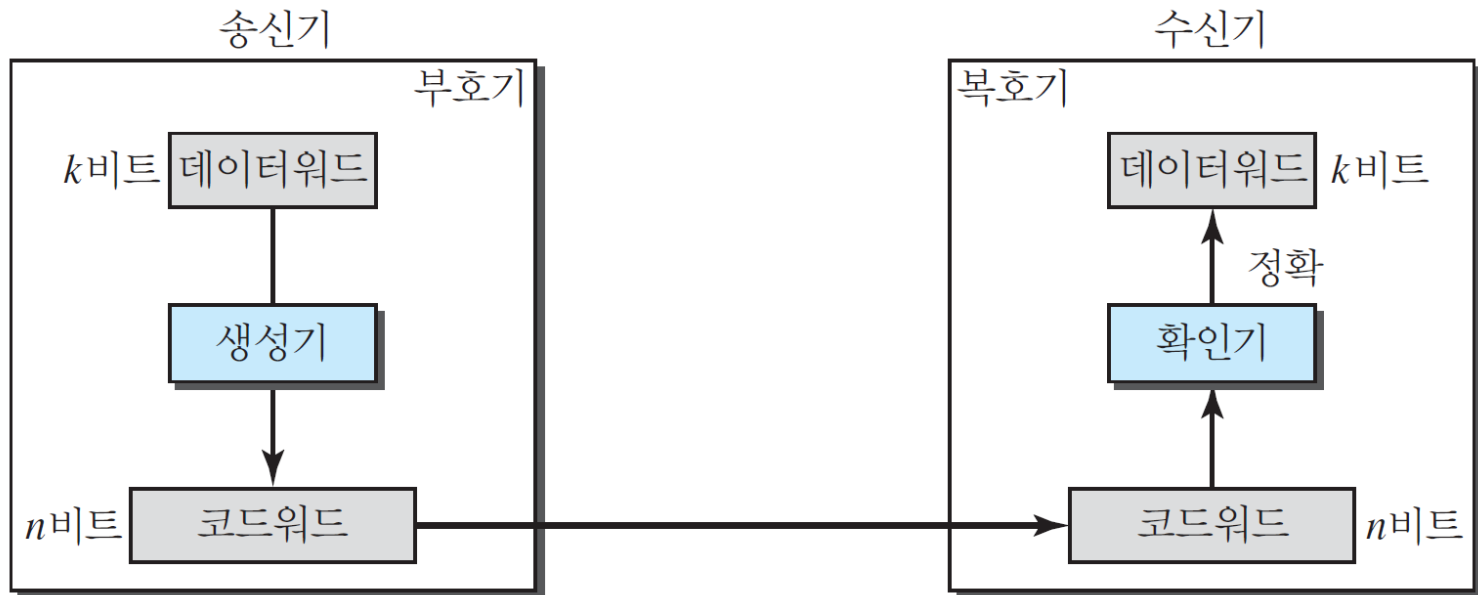
표 10.1 오류 검출(예제 10.2)을 위한 한 가지 코드

| 데이터워드 | 코드워드 | 데이터워드 | 코드워드 |
|-------|------|-------|------|
| 00    | 000  | 10    | 101  |
| 01    | 011  | 11    | 110  |

# 블록 부호화

## ■ 오류 정정

- ▶ 오류 검출보다 어렵다.
- ▶ 오류 검출보다 많은 중복 비트가 필요



### ▶ 예제 10.3

- 2비트 데이터워드에 3비트의 중복 비트를 넣어 5비트 코드워드를 만든다. 데이터워드가 01 이라고 하자. 송신자는 표를 보고 코드워드 01011을 알아낸다. 코드워드가 전송 도중 손상되어 01001이 수신.
- 수신된 코드워드를 첫 번째 코드워드와 비교하여(01001 대 00000), 수신자는 두 코드워드가 두 비트가 다르므로 첫 번째 코드워드를 보낸 것이 아니라고 추정
- 같은 이유로 원래의 코드워드는 3번째나 4번째의 코드워드가 될 수 없다.
- 원래의 코드워드는 표의 2번째 것일 수 밖에 없는데, 이는 수신된 코드워드와 1비트가 다른 것은 이것뿐이기 때문에 수신자는 01001을 01011로 바꾸어 넣고 표로부터 데이터워드가 01 인 것을 알게 된다.

표 10.2 오류 정정(예제 10.3)을 위한 코드 예

| 데이터워드 | 코드워드  | 데이터워드 | 코드워드  |
|-------|-------|-------|-------|
| 00    | 00000 | 10    | 10101 |
| 01    | 01011 | 11    | 11110 |

# 블록 부호화

## ■ 해밍 거리

### ▶ 오류 제어를 위해 중심 개념

- 두 워드 사이의 해밍 거리는 서로 다른 해당 비트의 개수
- 두 워드  $x$ 와  $y$  사이의 해밍 거리를  $d(x, y)$ 로 표시
- 두 워드에 XOR 연산을 해서 얻은 결과 값의 1의 개수
- $d(000, 011)$ 은 2인데 이는  $000 \oplus 011$ 은 011(2개의 1) 이기 때문.
- $d(10101, 11110)$ 은 3인데  $10101 \oplus 11110$ 은 01011(3개의 1)이기 때문

### ▶ 오류 검출을 위한 최소 해밍 거리( $d_{\min}$ )

- 모든 가능한 코드워드 쌍 사이의 가장 작은 해밍 거리

# 블록 부호화

## ▶ 예제 10.5

- 첫 번째 코드 기법(표 10.1)의 최소 해밍 거리는 2이다. 이 코드는 오직 단일 비트 오류를 검출할 수 있다.
- 예를 들면 세 번째 코드(101)가 전송되었는데 하나의 비트 오류가 발생했다면 수신된 코드는 그 어떤 유효 코드가 일치하지 않는다.
- 그러나 2개의 오류가 발생했다면 수신된 코드워드는 다른 유효 코드와 일치할 수 있으므로 오류를 알아낼 수 없다.

표 10.1 오류 검출(예제 10.2)을 위한 한 가지 코드

| 데이터워드 | 코드워드 | 데이터워드 | 코드워드 |
|-------|------|-------|------|
| 00    | 000  | 10    | 101  |
| 01    | 011  | 11    | 110  |



# 블록 부호화

## ▶ 예제 10.6

- 코드 기법에서 해밍 거리  $d_{\min} = 4$ 이다.
- 이 코드는 3개까지의 오류( $d = s + 1$  또는  $s = 3$ )를 검출할 수 있다.

# 블록 부호화

## ▶ 선형 블록 코드

- 오늘날 사용되는 대부분의 블록 코드는 선형 블록 코드(linear block codes) 중 하나
- 선형 블록 코드에서는 임의의 두 유효 코드워드에 XOR 연산을 가하면 다른 유효 코드워드가 생성된다.
- 비선형 블록 코드는 오류 검출이나 정정에 널리 사용되지 않음
  - ✓ 그 구조가 이론적으로 복잡하고 구현하기 힘들기 때문
- 선형 블록 코드의 최소 거리
  - ✓ 최소 해밍 거리는 0이 아닌 가장 적은 수의 1을 가지고 있는 코드워드의 1의 개수

# 블록 부호화

## ▶ 예제 10.8

- 첫 번째 코드(표 10.1)에서는 0이 아닌 코드워드의 1의 개수는 각각 2, 2 및 2이다. 그러므로 최소 해밍 거리  $d_{\min} = 2$ 이다.

표 10.1 오류 검출(예제 10.2)을 위한 한 가지 코드

| 데이터워드 | 코드워드 | 데이터워드 | 코드워드 |
|-------|------|-------|------|
| 00    | 000  | 10    | 101  |
| 01    | 011  | 11    | 110  |

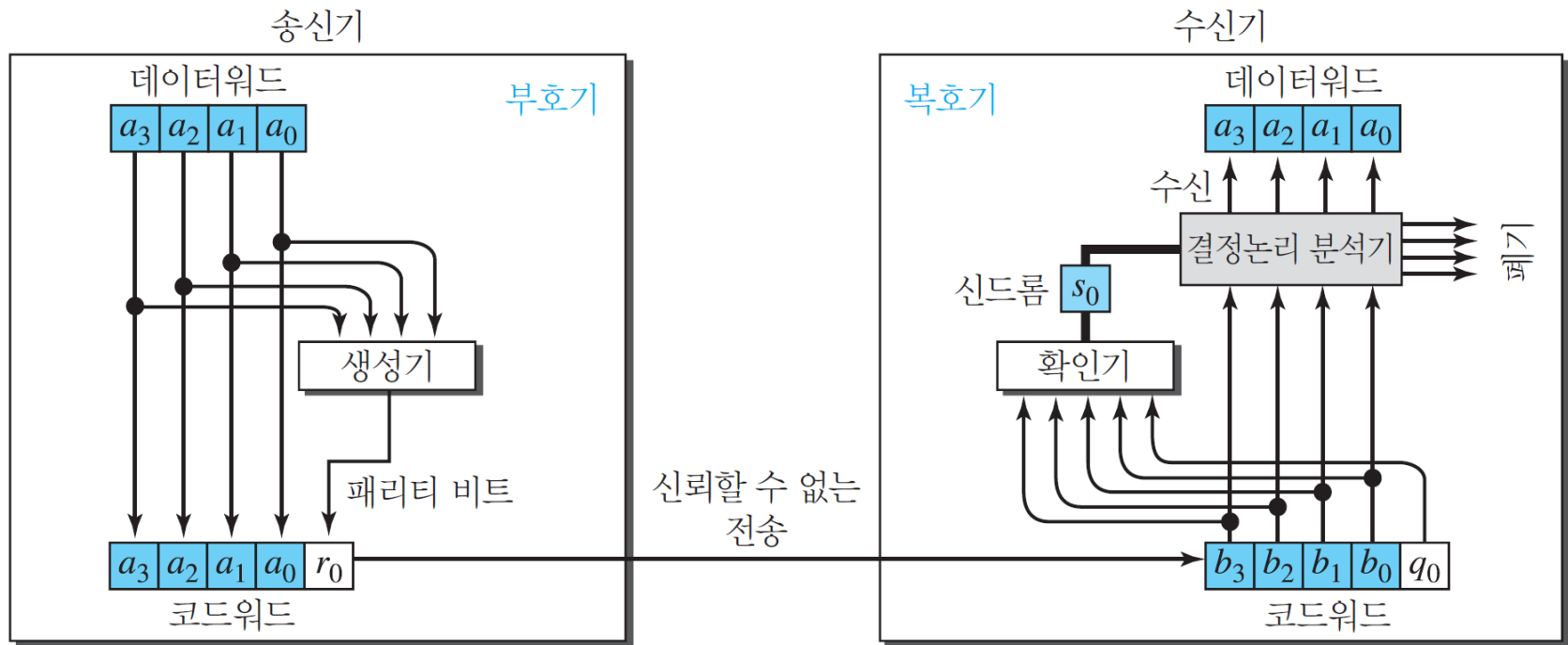
- 패리티 검사 코드
  - ✓  $k$ 비트 데이터워드를  $n = k + 1$ 이 되도록  $n$ 비트 코드워드로 바꾸는 것
  - ✓ 추가된 비트를 패리티 비트(parity bit)라고 함
  - ✓  $d_{\min} = 2$  이며, 단일 비트 오류 검출 코드라는 것을 의미
  - ✓ 패리티 검사 코드는 홀수 개의 오류를 검출

표 10.3 단순 패리티 검사 코드 C(5, 4)

| 데이터워드 | 코드워드          | 데이터워드 | 코드워드          |
|-------|---------------|-------|---------------|
| 0000  | 0000 <b>0</b> | 1000  | 1000 <b>1</b> |
| 0001  | 0001 <b>1</b> | 1001  | 1001 <b>0</b> |
| 0010  | 0010 <b>1</b> | 1010  | 1010 <b>0</b> |
| 0011  | 0011 <b>0</b> | 1011  | 1011 <b>1</b> |
| 0100  | 0100 <b>1</b> | 1100  | 1100 <b>0</b> |
| 0101  | 0101 <b>0</b> | 1101  | 1101 <b>1</b> |
| 0110  | 0110 <b>0</b> | 1110  | 1110 <b>1</b> |
| 0111  | 0111 <b>1</b> | 1111  | 1111 <b>0</b> |

# 선형 블록 코드

✓ 단순 패리티 검사 코드의 부호기와 복호기



# 선형 블록 코드

- 2차원 패리티 검사(two-dimensional parity check)

|       |   |   |   |   |   |   |   |          |
|-------|---|---|---|---|---|---|---|----------|
| 1     | 1 | 0 | 0 | 1 | 1 | 1 | 1 | 행<br>패리티 |
| 1     | 0 | 1 | 1 | 1 | 0 | 1 | 1 |          |
| 0     | 1 | 1 | 1 | 0 | 0 | 1 | 0 |          |
| 0     | 1 | 0 | 1 | 0 | 0 | 1 | 1 |          |
| 열 패리티 |   |   |   |   |   |   |   | 1        |

a. 행과 열 패리티

|   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|
| 1 | 1 | 0 | 0 | 1 | 1 | 1 | 1 | 1 |
| 1 | 0 | 1 | 1 | 1 | 0 | 1 | 1 | 1 |
| 0 | 1 | 1 | 1 | 0 | 0 | 1 | 0 | 0 |
| 0 | 1 | 0 | 1 | 0 | 0 | 1 | 1 | 1 |
|   |   |   |   |   |   |   |   | 1 |
| 0 | 1 | 0 | 1 | 0 | 1 | 0 | 1 | 1 |

b. 하나의 오류가 두 곳에 영향을 준다

|   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|
| 1 | 1 | 0 | 0 | 1 | 1 | 1 | 1 | 1 |
| 1 | 0 | 1 | 1 | 1 | 0 | 1 | 1 | 1 |
| 0 | 1 | 1 | 1 | 0 | 0 | 1 | 0 | 0 |
| 0 | 1 | 0 | 1 | 0 | 0 | 1 | 1 | 1 |
|   |   |   |   |   |   |   |   | 1 |
| 0 | 1 | 0 | 1 | 0 | 1 | 0 | 1 | 1 |

c. 두 개의 오류가 두 곳에 영향을 준다

|   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|
| 1 | 1 | 0 | 0 | 1 | 1 | 1 | 1 | 1 |
| 1 | 0 | 1 | 1 | 1 | 0 | 1 | 1 | 1 |
| 0 | 1 | 1 | 1 | 0 | 0 | 1 | 0 | 0 |
| 0 | 1 | 0 | 1 | 0 | 0 | 1 | 1 | 1 |
|   |   |   |   |   |   |   |   | 1 |
| 0 | 1 | 0 | 1 | 0 | 1 | 0 | 1 | 1 |

d. 3개의 오류가 4곳에 영향을 준다

|   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|
| 1 | 1 | 0 | 0 | 1 | 1 | 1 | 1 | 1 |
| 1 | 0 | 1 | 1 | 1 | 0 | 1 | 1 | 1 |
| 0 | 1 | 1 | 1 | 0 | 0 | 1 | 0 | 0 |
| 0 | 1 | 0 | 1 | 0 | 0 | 1 | 1 | 1 |
|   |   |   |   |   |   |   |   | 1 |
| 0 | 1 | 0 | 1 | 0 | 1 | 0 | 1 | 1 |

e. 4개의 오류를 검출하지 못한다

# 순환 코드

## ■ 순환 코드

- ▶ 하나의 특별한 성질을 갖는 선형 블록 코드
- ▶ 코드워드를 순환시키면 다른 코드워드를 얻음



# 순환 코드

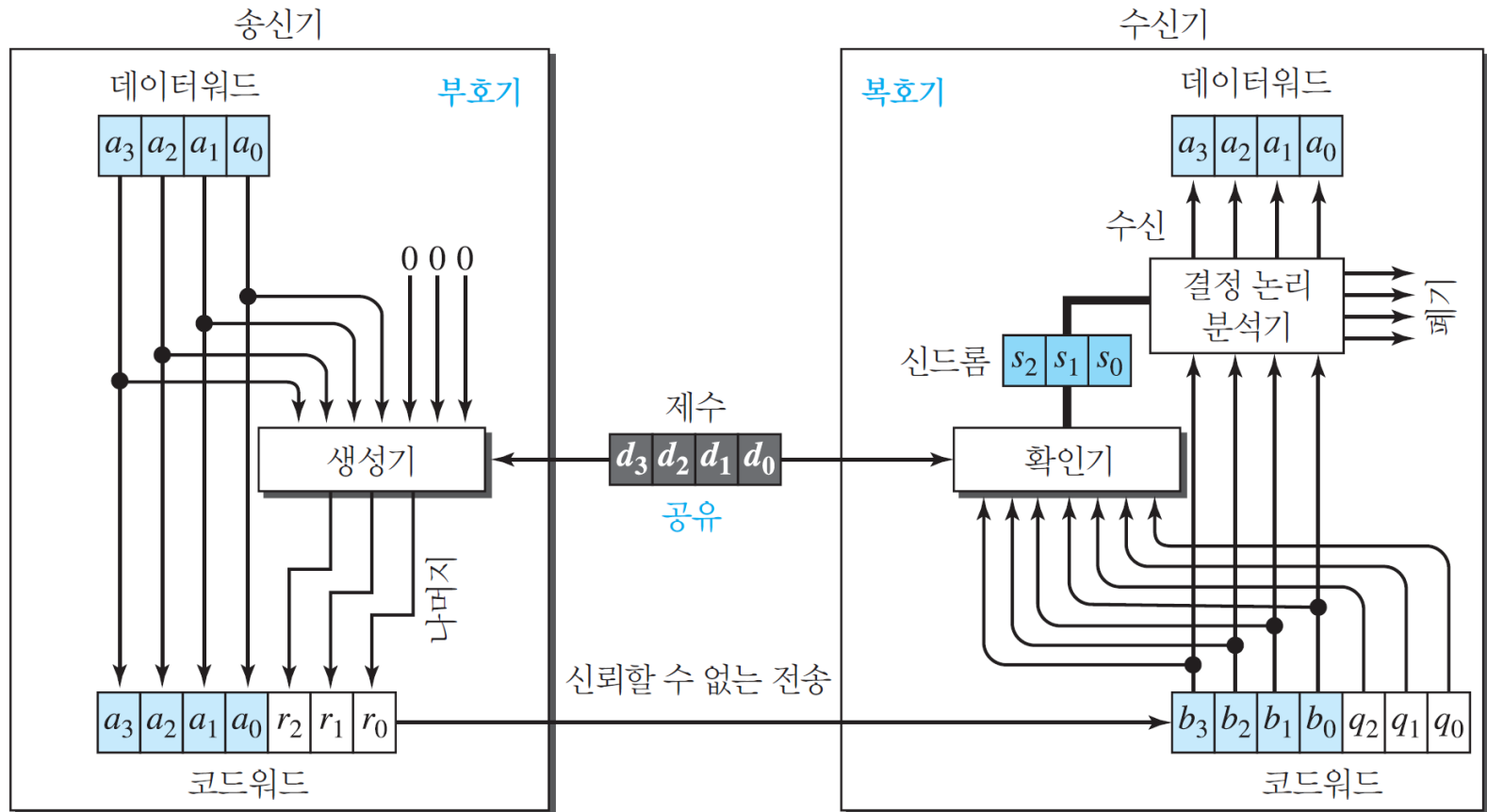
- 순환 중복 검사(CRC, Cyclic Redundancy Check)
  - ▶ 오류를 정정하기 위한 순환 코드를 생성
  - ▶ LAN이나 WAN에서 많이 사용

표 10.4 CRC 코드 C(7, 4)

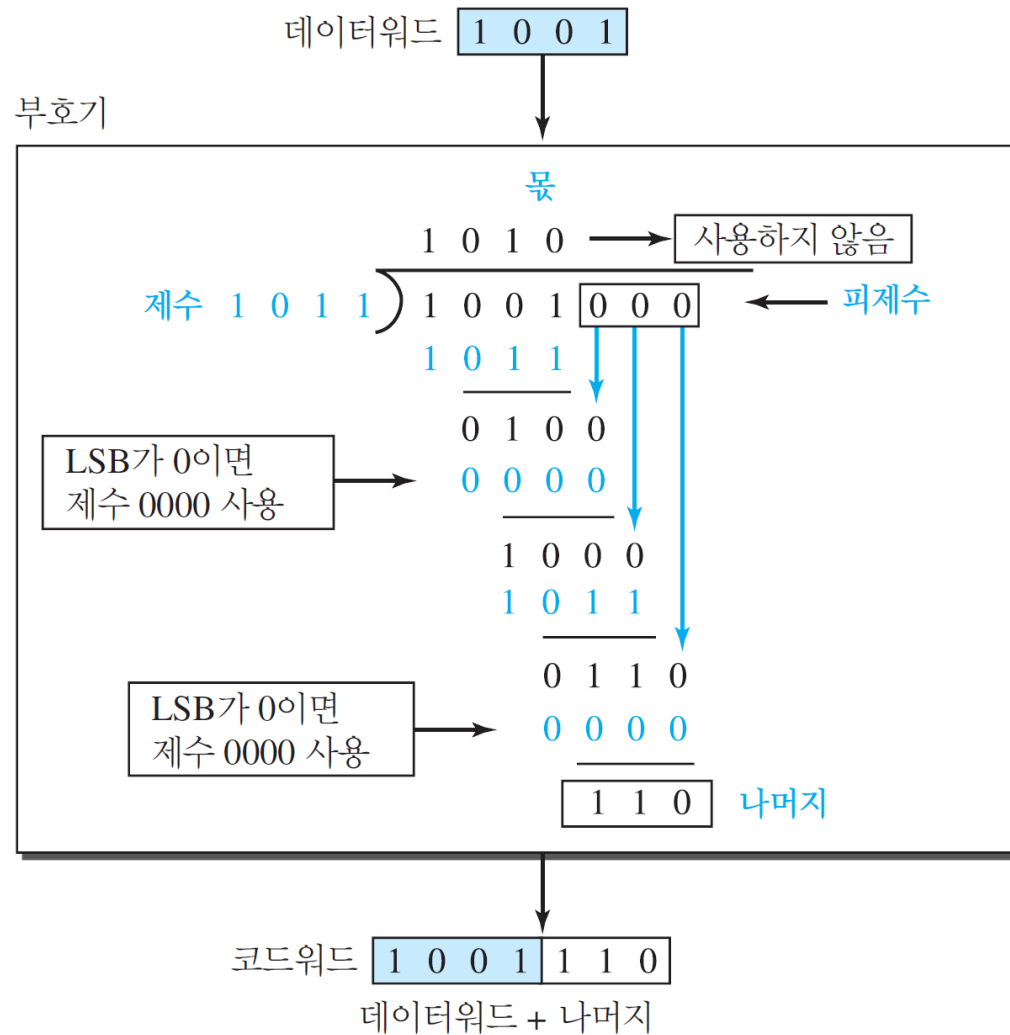
| 데이터워드 | 코드워드    | 데이터워드 | 코드워드    |
|-------|---------|-------|---------|
| 0000  | 0000000 | 1000  | 1000110 |
| 0001  | 0001101 | 1001  | 1001011 |
| 0010  | 0010111 | 1010  | 1010001 |
| 0011  | 0011010 | 1011  | 1011100 |
| 0100  | 0100011 | 1100  | 1100101 |
| 0101  | 0101110 | 1101  | 1101000 |
| 0110  | 0110100 | 1110  | 1110010 |
| 0111  | 0111001 | 1111  | 1111111 |

# 순환 코드

## ▶ CRC 부호기와 복호기



## ▶ CRC 부호기의 나눗셈

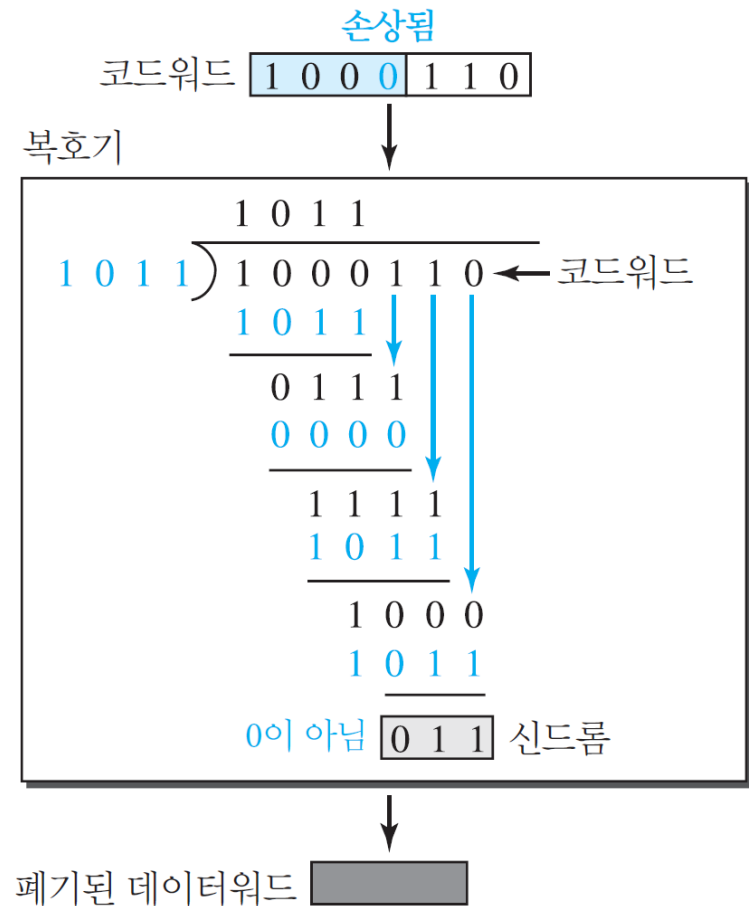
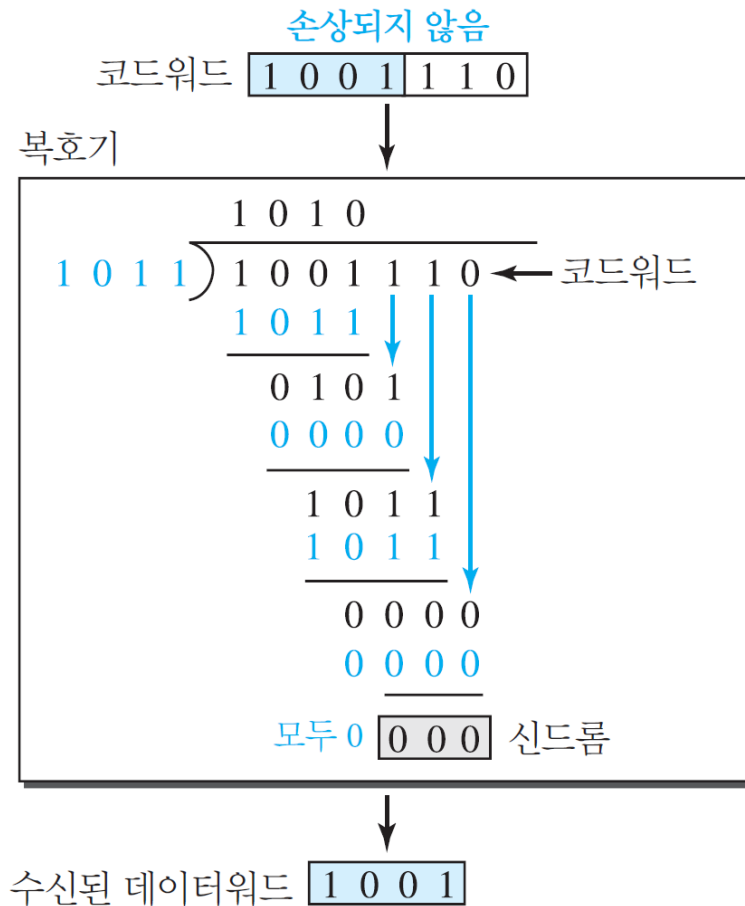


참고:

곱하기: AND

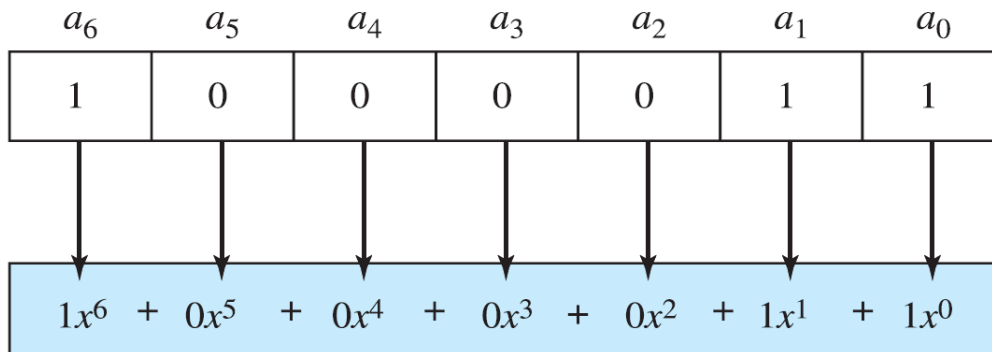
빼기: XOR

## ▶ CRC 복호기의 나눗셈

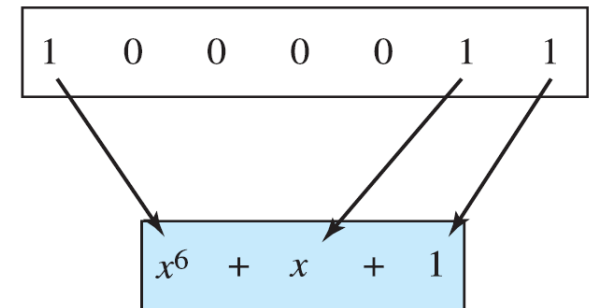


# 순환 코드

▶ 2진 워드를 표시하는 다항식



a. 2진 패턴과 다항식



b. 약식

# 순환 코드

## ■ 다항식

- ▶ 순환 코드를 이해하기 위해 다항식으로 표현
- ▶ 다항식의 차수
  - 다항식의 최고 지수
  - 예:  $x^6 + x + 1$ 의 차수는 6
- ▶ 다항식의 덧셈과 뺄셈
  - 모듈러 2 연산
  - 덧셈과 뺄셈의 결과는 같음
  - 덧셈, 뺄셈은 항목을 합하여 같은 차수의 항목을 제거하는 것
  - 예:  $x^5 + x^4 + x^2$ 과  $x^6 + x^4 + x^2$ 을 더하면  $x^6 + x^5$ 만이 남고, 항목  $x^4$  과  $x^2$ 는 제거

# 순환 코드

- ▶ 항목의 곱셈과 나눗셈
  - 곱셈은 지수를 더함
  - 예:  $x^3 \times x^4$ 은  $x^7$
  - 나눗셈은 첫 번째 항목의 지수에서 두 번째 항목의 지수를 뺀다
  - 예:  $x^5 / x^2$ 은  $x^3$ 이다.

$$\begin{aligned}(x^5 + x^3 + x^2 + x)(x^2 + x + 1) &= x^7 + x^6 + x^5 + x^5 + x^4 + x^3 + x^4 + x^3 + x^2 + x^3 + x^2 + x \\ &= x^7 + x^6 + x^3 + x\end{aligned}$$

# 순환 코드

## ▶ 이동

- 원편 이동: 오른쪽 비트에 0을 더하는 것
- 오른쪽 이동: 오른쪽 비트를 지우는 것

왼쪽으로 3비트 이동 :  $10011 \rightarrow 10011000$

$$x^4 + x + 1 \rightarrow x^7 + x^4 + x^3$$

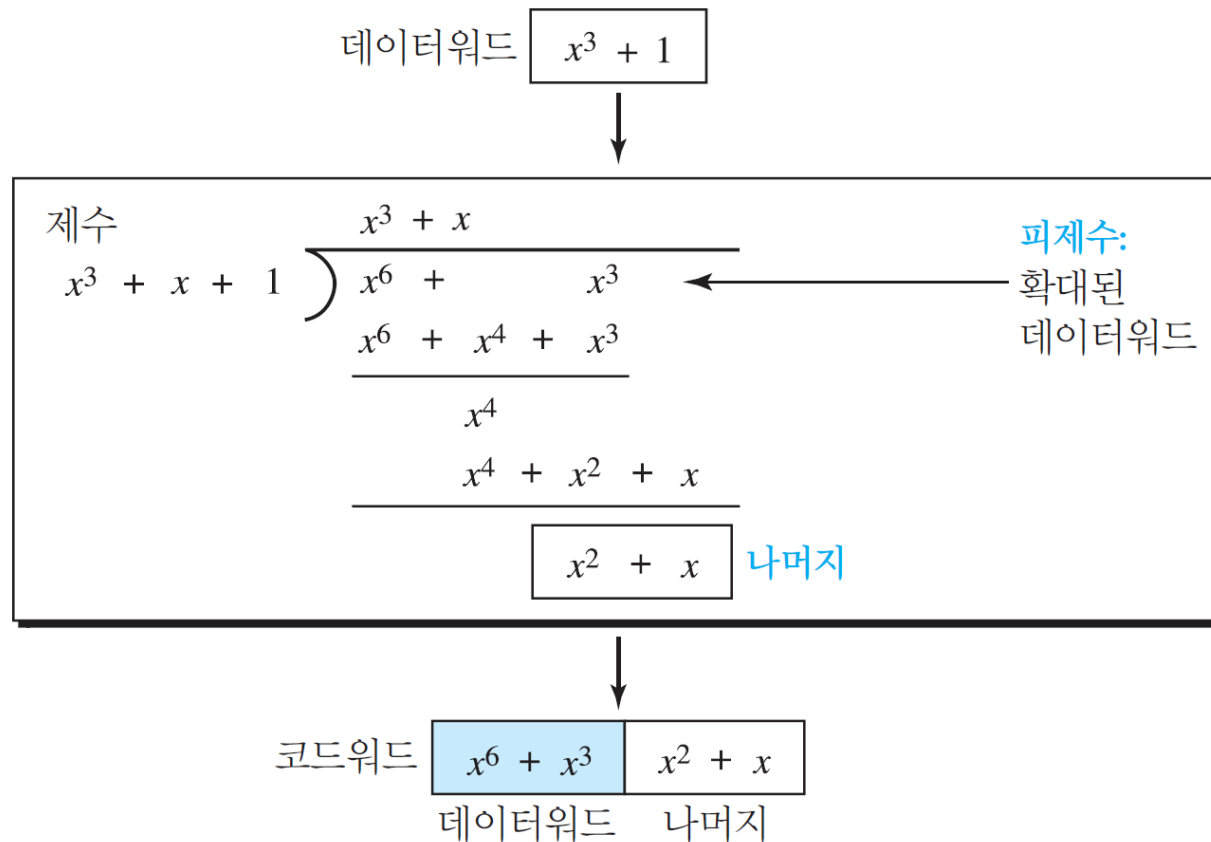
오른쪽으로 3비트 이동 :  $10011 \rightarrow 10$

$$x^4 + x + 1 \rightarrow x$$



## ■ 다항식을 사용하는 순환 코드 부호기

- ▶ 제수는 보통 생성 다항식 또는 생성식이라고 부름



# 순환 코드

## ▶ 표준 다항식

표 10.5 표준 다항식

| 이름     | 다항식                                                                                                                                                       | 응용         |
|--------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|------------|
| CRC-8  | $x^8 + x^2 + x + 1$<br><b>100000111</b>                                                                                                                   | ATM<br>헤더  |
| CRC-10 | $x^{10} + x^9 + x^5 + x^4 + x^2 + 1$<br><b>11000110101</b>                                                                                                | ATM<br>AAL |
| CRC-16 | $x^{16} + x^{12} + x^5 + 1$<br><b>10001000000100001</b>                                                                                                   | HDLC       |
| CRC-32 | $x^{32} + x^{26} + x^{23} + x^{22} + x^{16} + x^{12} + x^{11} + x^{10} + x^8 + x^7 + x^5 + x^4 + x^2 + x + 1$<br><b>100000100110000010001110110110111</b> | LAN        |

# 순환 코드

## ■ 순환 코드의 이점

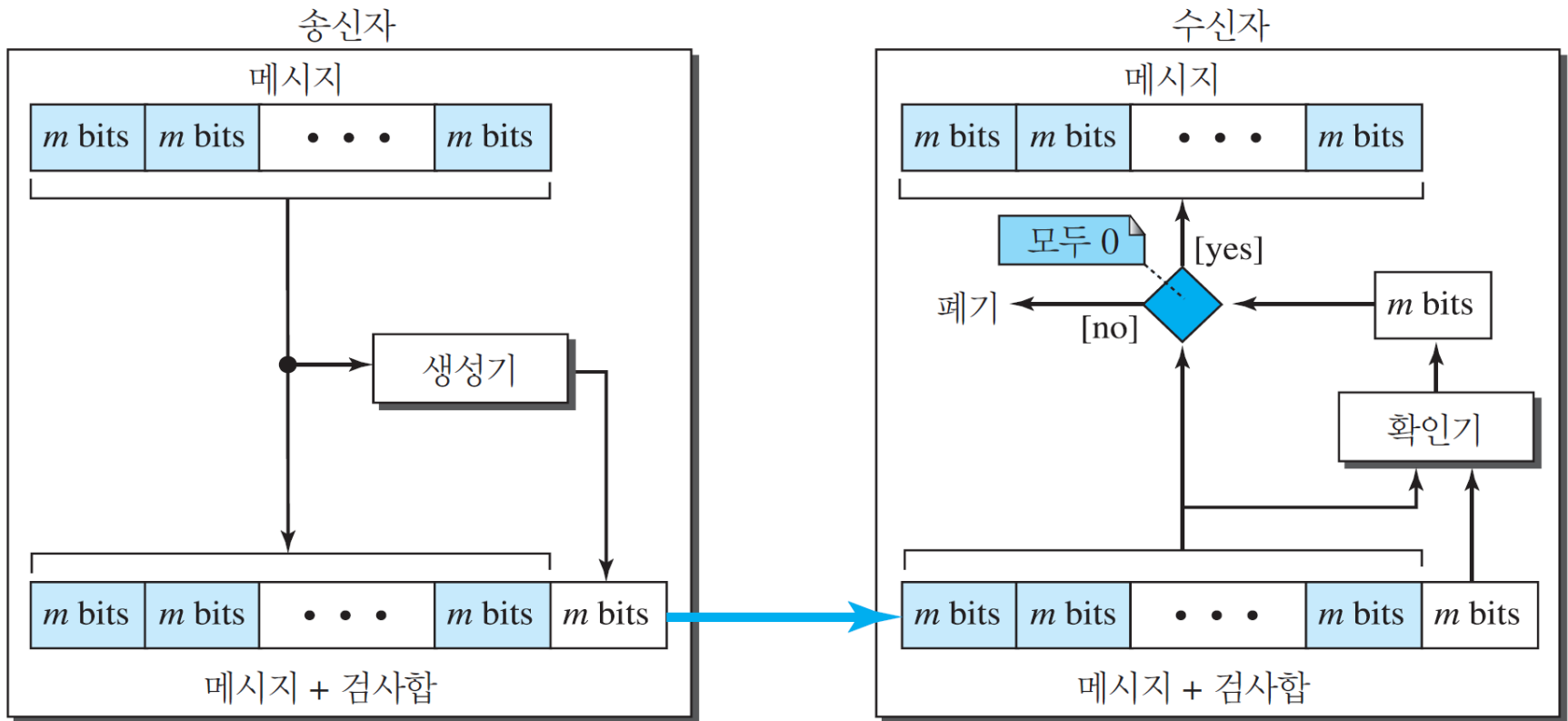
- ▶ 단일 비트, 2 비트, 홀수 개의 비트, 폭주 오류를 검출하는데 우수함
- ▶ 하드웨어 및 소프트웨어로 쉽게 구현
- ▶ 하드웨어로 구현시 속도가 빠름
  - 많은 네트워크에서 사용

# 검사합

## ■ 검사합

- ▶ 모든 길이의 메시지에 적용시킬 수 있는 오류 검출 기법
- ▶ 네트워크층과 전송층에서 사용
- ▶ 생성기(generator)
  - 그 다음 검사합이라는 메시지와 함께 전송되는 별도의 m비트 단위를 생성
- ▶ 확인기(checker)
  - 메시지와 전송된 검사합을 이용하여 새로운 검사합을 생성
  - 새로운 검사합이 모두 0이면, 메시지 승인
  - 그렇지 않으면, 메시지는 폐기

# 검사합



# 검사합

## ■ 개념

### ▶ 예제 10.10

- 메시지가 5개의 4비트 수라고 하자.
- 이 숫자들을 전송하는 것 외에 전체 숫자의 합도 전송한다
- 예를들면 숫자들이 (7, 11, 12, 0, 6)이라면 우리는 (7, 11, 12, 0, 6, 36)을 전송하는데 36은 원래 숫자들의 합이다.
- 수신자는 5개의 숫자를 합하여 그 결과를 합과 비교한다. 두 값이 같으면 수신자는 오류가 없는 것으로 가정하여 5개의 숫자를 받아들이고 합은 버린다.
- 그렇지 않으면 오류가 있는 것이어서 데이터를 받아들이지 않는다.

# 검사합

## ▶ 1의 보수의 덧셈

- 앞의 예는 한 가지 단점이 있다.
- 검사합을 제외한 모든 데이터는 4비트 워드(15보다 작다)이다. 해결책으로는 1의 보수(one's complement) 연산을 사용하는 것
- 이 연산에서는 0과  $2^m - 1$  사이의 부호 없는 수를 m비트만을 사용하여 나타낸다.
- 수가 m비트보다 많으면 왼쪽의 남은 비트들은 m개의 오른쪽 비트들에 더해진다.

# 검사합

## ▶ 예제 10.11

- 이전의 예에서, 10진수 36은 2진수로  $(100100)_2$ 이다. 오른쪽 4비트와 나머지 왼쪽 비트를 더해서 4비트의 숫자로 변환한다

$$(10)_2 + (0100)_2 = (0110)_2 \rightarrow (6)_{10}$$

- 합으로 36을 전송하는 대신에 6을 전송할 수 있다. (7, 11, 12, 0, 6, 6) 수신자는 처음 5개의 숫자에 1의 보수 연산을 추가할 수 있다.
- 만약 결과가 6이라면 숫자들은 받아들여지고, 그렇지 않으면 버려진다.

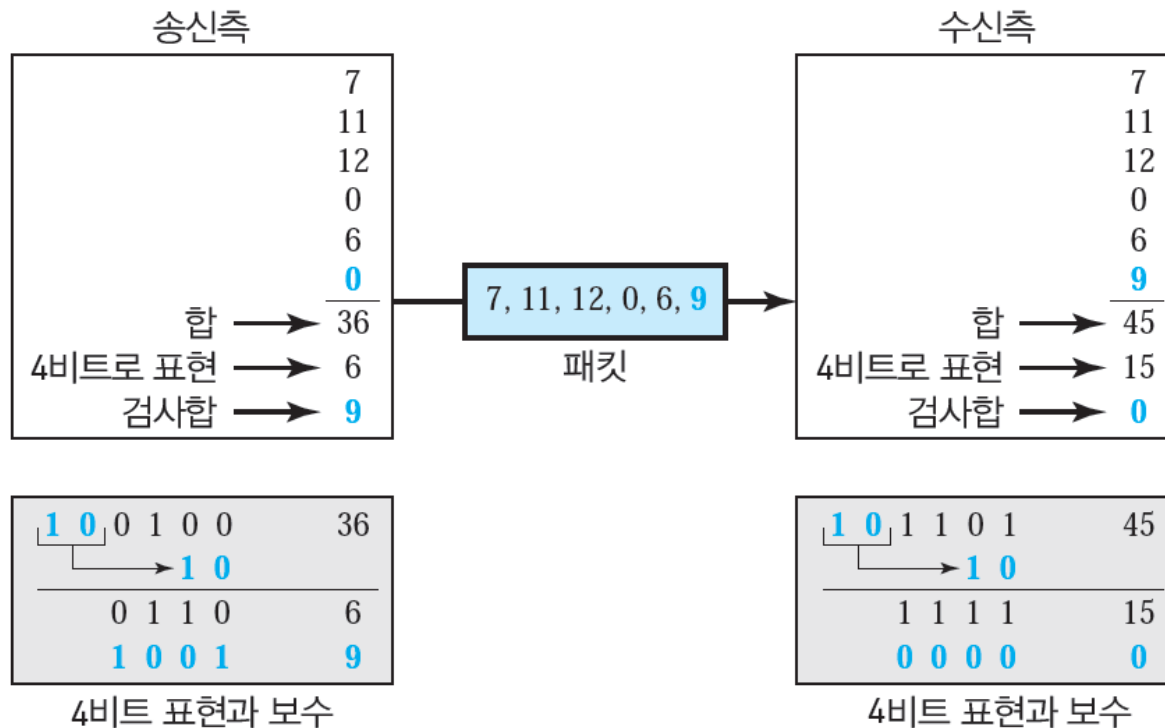


# 검사합

- ▶ 검사합
  - 보수의 합인 검사합을 전송하면 수신자의 작업을 보다 쉽게 만들 수 있음

# 검사합

## ▶ 예제 10.12



# 검사합

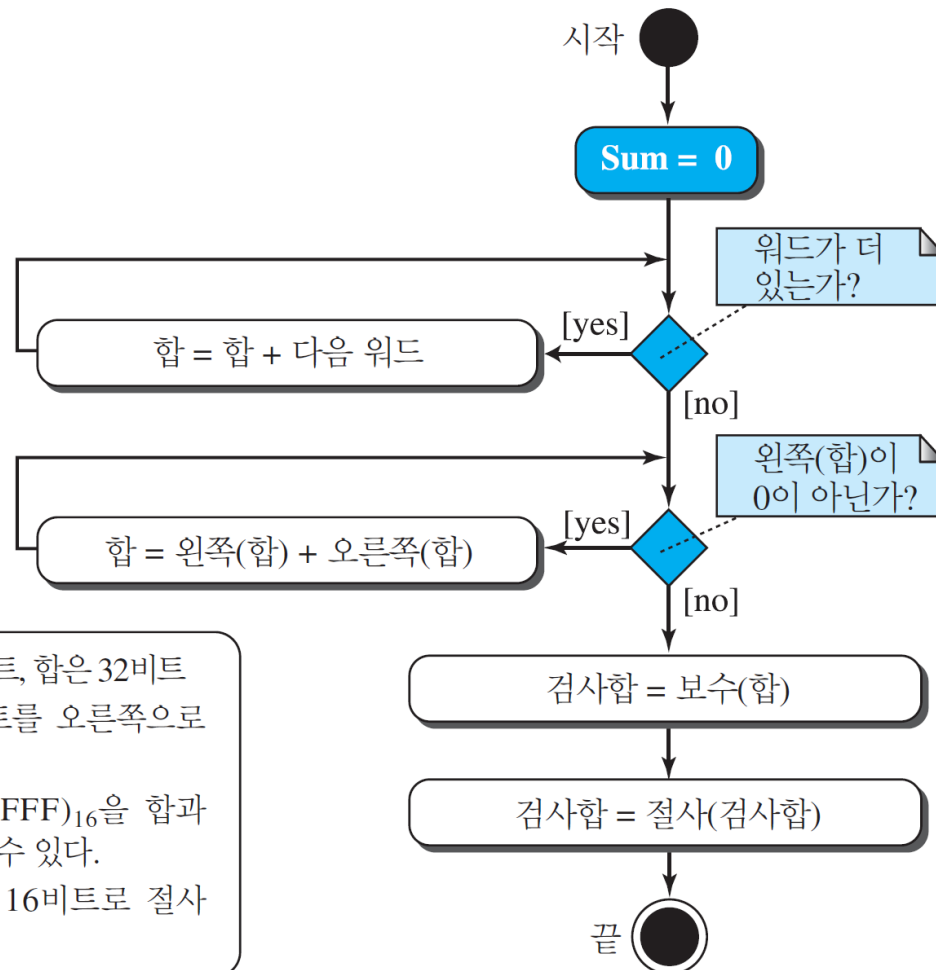
## ▶ 인터넷 검사합

표 10.6 일반적인 검사합 계산 절차

| 송신자                                                                                                                                                                                                                        | 수신자                                                                                                                                                                                                                         |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"><li>1. 메시지를 16비트 워드로 나눈다.</li><li>2. 검사합 워드의 값은 처음에 0으로 둔다.</li><li>3. 검사합을 포함한 모든 워드를 1의 보수 연산을 하여 더한다.</li><li>4. 그 합을 보수를 취하여 그 최종값을 검사합으로 한다.</li><li>5. 검사합을 데이터와 함께 보낸다.</li></ul> | <ul style="list-style-type: none"><li>1. 메시지와 검사합을 전송받는다.</li><li>2. 메시지를 16비트 워드로 나눈다.</li><li>3. 모든 워드를 1의 보수 연산을 하여 더한다.</li><li>4. 그 합을 보수를 취하여 그 최종값을 새로운 검사합으로 한다.</li><li>5. 검사합의 값이 0이면 메시지를 받고 아니면 폐기한다.</li></ul> |

# 검사합

## ▶ 알고리즘



### 참고:

- 워드와 검사합은 16비트, 합은 32비트
- 왼쪽(합)은 합 16비트를 오른쪽으로 이동하면 알 수 있다.
- 오른쪽(합)은  $(000FFFF)_{16}$ 을 합과 AND 연산으로 구할 수 있다.
- 검사합을 찾은 후에 16비트로 절사한다.

# 검사합

## ▶ 예제

- 8개의 문자로 된 Forouzan의 검사합을 계산해 보자.

| 1 | 0 | 1 | 3 | 자리 올림    |
|---|---|---|---|----------|
| 4 | 6 | 6 | F | (Fo)     |
| 7 | 2 | 6 | F | (ro)     |
| 7 | 5 | 7 | A | (uz)     |
| 6 | 1 | 6 | E | (an)     |
| 0 | 0 | 0 | 0 | 검사합 (초기) |
| 8 | F | C | 6 | 합 (부분)   |
|   |   |   | 1 |          |
| 8 | F | C | 7 | 합        |
| 7 | 0 | 3 | 8 | 검사합 (보냄) |

a. 송신측 검사합

| 1 | 0 | 1 | 3 | 자리 올림       |
|---|---|---|---|-------------|
| 4 | 6 | 6 | F | (Fo)        |
| 7 | 2 | 6 | F | (ro)        |
| 7 | 5 | 7 | A | (uz)        |
| 6 | 1 | 6 | E | (an)        |
| 7 | 0 | 3 | 8 | 검사합(수신)     |
| F | F | F | E | 합(부분)       |
|   |   |   | 1 |             |
| F | F | F | F | 합           |
| 0 | 0 | 0 | 0 | 검사합 (새로운 값) |

b. 수신측 검사합